

eVrtić sistem

Strukturalni dizajn paterni

Strukturalni paterni su korisni u eVrtić sistemu zato što postojeći dijagram klasa već sadrži više međusobno povezanih modela: korisničke uloge, djecu, grupe, dnevne izvještaje, evidenciju dolaska i odlaska, dnevni QR kod, obavijesti i sedmične ili mjesečne sažetke aktivnosti. Kako sistem raste, iste klase bi mogle preuzeti previše odgovornosti: Obavijest bi morala znati detalje svakog kanala slanja, DnevniIzvještaj bi morao pokrivati sve moguće dodatke izvještaju, a klase koje pristupaju podacima djeteta morale bi ponavljati sigurnosne provjere. Uvođenjem strukturalnih paterna postojeće klase ostaju jasne, a nove funkcionalnosti se dodaju kroz pomoćne klase, interfejsa i posrednike koji se prirodno nadovezuju na obični dijagram klasa.

Pregled problema i predloženih paterna

Problem u postojećem dijagramu klasa	Rizik bez paterna	Patern koji pomaže
Klasa Obavijest ima kanalSlanja, ali različiti kanali mogu imati različit način slanja.	Dupliranje logike za email, aplikacijske i buduće kanale slanja.	Adapter / Bridge
Dnevni izvještaj, evidencija dolaska/odlaska, QR kod i obavijest često učestvuju u istoj korisničkoj akciji.	Jedna operacija bi zahtijevala ručno povezivanje više klasa i veći rizik od greške.	Facade
Dnevni izvještaji nekad imaju samo osnovne podatke, a nekad dodatne napomene, alergije, upozorenja ili validaciju.	Potrebno bi bilo praviti mnogo izvedenih klasa za svaku kombinaciju izvještaja.	Decorator
Roditelj, odgajatelj i administrator imaju različita prava pristupa podacima djeteta.	Sigurnosna pravila bi se ponavljala na više mjesta i lakše bi se napravila greška.	Proxy
Sažetak aktivnosti nastaje iz više dnevnih izvještaja i manjih stavki aktivnosti.	SazetakAktivnosti bi morao posebno obrađivati svaki tip stavke.	Composite
U sistemu se ponavljaju isti statusi, tipovi sažetka, kanali slanja i šabloni prikaza.	Nepotrebno dupliranje podataka i šablona kroz više objekata.	Flyweight

1. Adapter patern

Namjena paterna: Adapter omogućava da se postojeća klasa ili vanjski servis koristi kroz interfejs koji odgovara ostatku sistema, bez izmjene postojeće klase.

Primjena u sistemu eVrtić: U eVrtić sistemu adapter se može primijeniti kod slanja obavijesti i rada sa QR kodom. Postojeća klasa Obavijest već sadrži kanalSlanja i statusObavijesti, dok DnevniQRCode i EvidencijaDolaskaOdlaska učestvuju u procesu evidentiranja dolaska i odlaska. Ako se naknadno uvede vanjski email servis, push servis ili čitač QR koda, ti servisi mogu imati drugačiji način pozivanja. Adapter bi njihove različite metode prilagodio zajedničkom obliku koji ostatak sistema koristi na isti način.

Zašto je potreban: Bez adaptera bi se u klasi Obavijest ili u pomoćnoj logici slanja pojavljivali posebni uslovi za svaki kanal slanja. To bi otežalo održavanje i proširenje sistema. Adapter čuva postojeće klase Obavijest, DnevniQRCode i EvidencijaDolaskaOdlaska, a nove servise dodaje kao odvojene klase.

Veza sa običnim dijagramom klasa: Adapter se nadovezuje na klase Obavijest, KanalSlanja, StatusObavijesti, DnevniQRCode i EvidencijaDolaskaOdlaska. U običnom dijagramu se može dodati interfejs za slanje obavijesti i posebne adapter klase za konkretne načine slanja, bez promjene postojećih modela.

Predložene dopune dijagrama:

- IObavijestSender - zajednički interfejs sa metodom posalji(Obavijest).
- EmailObavijestAdapter - prilagođava vanjski email servis postojećoj klasi Obavijest.
- AplikacijskaObavijestAdapter - prilagođava slanje obavijesti unutar aplikacije.
- QRReaderAdapter - podatke dobijene skeniranjem QR koda pretvara u EvidencijaDolaskaOdlaska zapis.

Efekat na postojeći sistem: Sistem dobija jedinstven način komunikacije sa različitim vanjskim servisima, a obični dijagram klasa se proširuje malim brojem pomoćnih adapter klasa.

2. Facade patern

Namjena paterna: Facade patern daje jednostavan interfejs prema složenijem skupu klasa i podsistema.

Primjena u sistemu eVrtić: U eVrtić sistemu više model klasa često učestvuje u jednoj funkcionalnosti. Na primjer, kada se evidentira dnevna aktivnost djeteta, potrebno je povezati Dijete, Dnevnilzvjestaj, StatusObroka, EvidencijaDolaskaOdlaska i Obavijest. Kada roditelj koristi QR kod za dolazak ili odlazak, učestvuju DnevniQRCode, EvidencijaDolaskaOdlaska, Roditelj i Dijete. Umjesto da se svaka operacija ručno povezuje kroz više klasa, facade može ponuditi jednu jasnu metodu za cijeli proces.

Zašto je potreban: Facade je potreban jer smanjuje međusobnu zavisnost postojećih klasa. Klase poput Dnevnilzvjestaj, Obavijest i SazetakAktivnosti ostaju fokusirane na svoje podatke i osnovna ponašanja, dok facade koordinira redoslijed akcija i pravila poslovnog procesa.

Veza sa običnim dijagramom klasa: Facade se u običnom dijagramu klasa može dodati kao sloj pomoćnih klasa koje koriste postojeće klase Dijete, Roditelj, Odgajatelj, Dnevnilzvjestaj, Obavijest, DnevniQRCode, EvidencijaDolaskaOdlaska i SazetakAktivnosti. Ne mijenja se osnovna struktura modela, nego se uvodi jednostavniji ulaz u složen proces.

Predložene dopune dijagrama:

- DnevnaAktivnostFacade - objedinjuje kreiranje i ažuriranje dnevnih aktivnosti djeteta.
- PrisustvoFacade - koordinira QR kod, validaciju, dolazak i odlazak djeteta.
- IzvjestajFacade - priprema dnevni izvještaj za pregled roditelja ili odgajatelja.
- SazetakFacade - pokreće generisanje sedmičnog ili mjesečnog sažetka aktivnosti.

Efekat na postojeći sistem: Smanjuje se broj direktnih veza između postojećih klasa i dobija se jasnije mjesto za poslovna pravila koja uključuju više klasa odjednom.

3. Decorator patern

Namjena paterna: Decorator omogućava dinamičko dodavanje novih funkcionalnosti objektu bez mijenjanja njegove osnovne klase.

Primjena u sistemu eVrtić: U eVrtić sistemu osnovni Dnevnilzvjestaj može sadržavati datum, podatke o obroku, spavanju, vremenu dolaska i odlaska, raspoloženju i napomenama. Međutim, u praksi se izvještaju nekad dodaju posebna upozorenja: alergija, bolest, komentar odgajatelja, dodatna napomena o obroku ili informacija da je izvještaj validiran. Decorator omogućava da se takvi dodaci dodaju po potrebi, bez kreiranja velikog broja posebnih izvedenih klasa.

Zašto je potreban: Decorator je potreban zato što se sadržaj dnevnog izvještaja mijenja od djeteta do djeteta i od dana do dana. Ako bi se za svaku kombinaciju pravila posebna klasa, obični dijagram klasa bi postao nepregledan.

Veza sa običnim dijagramom klasa: Decorator se direktno veže za Dnevnilzvjestaj i za podatke koje izvještaj već sadrži: StatusObroka, napomenaAktivnosti, vrijemeDolaska, vrijemeOdlaska i validiran. Dodaci se uvode kao klase koje obavijaju osnovni izvještaj i proširuju njegov prikaz ili sadržaj.

Predložene dopune dijagrama:

- IzvjestajComponent - zajednički interfejs za osnovni izvještaj i dekoratore.
- OsnovniDnevnilzvjestaj - predstavlja postojeću klasu Dnevnilzvjestaj u obliku komponente.
- NapomenaDecorator - dodaje dodatnu napomenu odgajatelja.
- AlergijaDecorator - dodaje upozorenje za alergiju ili zdravstvenu napomenu.
- ValidacijaDecorator - dodaje oznaku ili poruku da je izvještaj validiran.

Efekat na postojeći sistem: Osnovna klasa Dnevnilzvjestaj ostaje jednostavna, a dodatni sadržaji se uvode fleksibilno i bez narušavanja postojećih veza u dijagramu.

4. Bridge patern

Namjena paterna: Bridge razdvaja apstrakciju od implementacije tako da se obje strane mogu mijenjati nezavisno.

Primjena u sistemu eVrtić: U eVrtić sistemu najprirodnija primjena Bridge paterna je kod obavijesti. Postoje različite vrste obavijesti: dnevna napomena, obavijest o dolasku ili odlasku, obavijest o sažetku i hitna obavijest. Istovremeno postoje različiti kanali slanja: aplikacija i email, a u budućnosti mogu postojati SMS ili push notifikacije. Bridge odvaja tip obavijesti od kanala slanja.

Zašto je potreban: Bez Bridge paterna broj klasa brzo raste, jer bi za svaku kombinaciju tipa obavijesti i kanala slanja trebalo praviti posebnu klasu. Bridge omogućava da se doda novi kanal bez mijenjanja svih vrsta obavijesti, ili nova vrsta obavijesti bez mijenjanja svih kanala.

Veza sa običnim dijagramom klasa: U običnom dijagramu klasa već postoji Obavijest, atribut kanalSlanja i enumeracija KanalSlanja. Bridge bi ovu strukturu proširio tako da Obavijest ima referencu na interfejs kanala slanja, a konkretni kanali predstavljaju odvojenu implementacionu hijerarhiju.

Predložene dopune dijagrama:

- ApstraktnaObavijest - apstrakcija koja sadrži referencu na IKanalSlanja.
- DnevnaNapomenaObavijest, HitnaObavijest i SazetakObavijest - konkretne vrste obavijesti.
- I KanalSlanja - implementacioni interfejs za slanje poruke.
- EmailKanal, AplikacijaKanal i SMSKanal - konkretne implementacije kanala slanja.

Efekat na postojeći sistem: Tip obavijesti i način slanja više nisu čvrsto spojeni. Time se čuva postojeća klasa Obavijest, a sistem se lakše proširuje novim kanalima ili vrstama poruka.

5. Proxy patern

Namjena paterna: Proxy omogućava pristup stvarnom objektu preko posrednika koji može kontrolisati pristup, odgoditi učitavanje ili izvršiti dodatne provjere.

Primjena u sistemu eVrtić: U eVrtić sistemu proxy je posebno važan zbog zaštite podataka djece. Roditelj smije vidjeti samo podatke svog djeteta, odgajatelj smije vidjeti podatke djece iz svoje grupe, a administrator ima šira prava za upravljanje profilima. Proxy klasa može stajati između korisnika sistema i stvarnih objekata kao što su Dijete, Dnevnilzvjestaj, SazetakAktivnosti i EvidencijaDolaskaOdlaska.

Zašto je potreban: Proxy je potreban jer sigurnosna pravila ne treba ponavljati u svakoj klasi koja koristi podatke djeteta. Ako se provjera prava pristupa izdvoji u proxy, smanjuje se mogućnost greške i lakše se održava pravilo da korisnik vidi samo podatke za koje ima dozvolu.

Veza sa običnim dijagramom klasa: Proxy se u običnom dijagramu veže za Korisnik, Roditelj, Odgajatelj, Administrator, Dijete, Grupa, Dnevnilzvjestaj i SazetakAktivnosti. Posebno koristi postojeće attribute uloga, statusNaloga i veze roditelj-dijete te odgajatelj-grupa.

Predložene dopune dijagrama:

- IDijetePodaciService - interfejs za pristup podacima djeteta.
- DijetePodaciService - stvarni servis koji radi sa objektima Dijete, Dnevnilzvjestaj i SazetakAktivnosti.
- DijetePodaciProxy - provjerava vezu roditelj-dijete ili odgajatelj-grupa prije vraćanja podataka.

- KorisnikProxy - provjerava status naloga prije dozvole za prijavu ili uređivanje profila.

Efekat na postojeći sistem: Sigurnosna pravila se izdvajaju u posebne posredničke klase, dok postojeće klase iz dijagrama ostaju fokusirane na domenske podatke.

6. Composite patern

Namjena paterna: Composite omogućava da se pojedinačni objekti i grupe objekata tretiraju na isti način, najčešće u strukturi stabla.

Primjena u sistemu eVrtić: U eVrtić sistemu dnevni izvještaj se prirodno sastoji od više manjih dijelova: obrok, spavanje, dolazak, odlazak, napomena i eventualni komentar odgajatelja. Sedmični ili mjesečni sažetak se zatim sastoji od više dnevnih izvještaja. Composite bi omogućio da se pojedinačna aktivnost, dnevni izvještaj i periodični sažetak obrađuju kroz isti interfejs.

Zašto je potreban: Composite je potreban jer olakšava generisanje sažetaka. Ako svaka stavka izvještaja zna kako da se prikaže, validira ili doda u sažetak, onda SazetakAktivnosti ne mora posebno provjeravati svaki mogući tip aktivnosti.

Veza sa običnim dijagramom klasa: Composite se veže za Dnevnilzvjestaj, EvidencijaDolaskaOdlaska i SazetakAktivnosti. Postojeća veza da jedan sažetak agregira više dnevnih zapisa može se modelovati kao kompozicija u kojoj su pojedinačne aktivnosti listovi, a dnevni i periodični izvještaji složene komponente.

Predložene dopune dijagrama:

- I AktivnostComponent - interfejs sa metodama prikazi(), validiraj() i dodajUSazetak().
- ObrokAktivnost, SpavanjeAktivnost, DolazakOdlazakAktivnost i NapomenaAktivnost - pojedinačne komponente.
- DnevnilzvjestajComposite - kompozicija više aktivnosti za jedan dan.
- PeriodicniSazetakComposite - kompozicija više dnevnih izvještaja za sedmicu ili mjesec.

Efekat na postojeći sistem: Sazeci i izvještaji se mogu graditi hijerarhijski, a obični dijagram klasa dobija jasniji prikaz odnosa između pojedinačnih aktivnosti i složenih izvještaja.

7. Flyweight patern

Namjena paterna: Flyweight smanjuje potrošnju memorije tako što se zajednički, nepromjenjivi podaci dijele između većeg broja objekata.

Primjena u sistemu eVrtić: U eVrtić sistemu veliki broj objekata može koristiti iste vrijednosti: StatusNaloga, StatusObroka, TipDogađaja, StatusEvidencije, TipSazetka, KanalSlanja i StatusObavijesti. Osim toga, obavijesti i izvještaji često koriste slične tekstualne šablone, na primjer obavijest o dolasku, odlasku ili dnevnoj napomeni. Flyweight omogućava da se zajednički statusi i šabloni čuvaju jednom i dijele između više konkretnih objekata.

Zašto je potreban: Bez Flyweight paterna isti tekstovi, šabloni i statusne vrijednosti mogli bi se nepotrebno ponavljati u velikom broju dnevnih izvještaja, evidencija i obavijesti. To povećava memorijsko zauzeće i otežava izmjenu zajedničkih pravila prikaza.

Veza sa običnim dijagramom klasa: U običnom dijagramu klasa već postoje brojne enumeracije i ponavljajući atributi. Flyweight bi se najviše vezao za Obavijest, Dnevnilzvjestaj, SazetakAktivnosti i statusne/tipske enumeracije. Umjesto da svaki objekat nosi puni šablon, objekat bi imao referencu na dijeljeni šablon ili tip.

Predložene dopune dijagrama:

- StatusFlyweightFactory - vraća dijeljene objekte za statuse i tipove događaja.
- ObavijestTemplateFlyweight - čuva zajedničke šablone obavijesti.
- IzvjestajTemplateFlyweight - čuva zajedničke šablone dnevnog izvještaja ili sažetka.
- FlyweightFactory - centralno mjesto za dohvat postojećih dijeljenih objekata.

Efekat na postojeći sistem: Smanjuje se dupliranje zajedničkih vrijednosti, a izmjena šablona obavijesti ili izvještaja može se obaviti na jednom mjestu.

